المزيد من الوراثة (More Inheritance)

1 المشكلة — لماذا نحتاج الكلاس الأساسي الافتراضي؟

تخيّل هذا الوضع: عندك **جد** (`grandparent`)، يرث منه **أب وأم** (`parent1`, `parent2`)، ثم **الطفل** (`child`) يرث من الأب والأم معاً. المشكلة أن الطفل سيرث **نسختيْن** من أعضاء الجد — واحدة عبر الأب وواحدة عبر الأم، وهذا **تكرار (Duplication)** يُسبب أخطاء

```text
        grandparent
       /            \
    parent1         parent2
       \            /
          child
```

بدون `virtual`: الطفل يحمل **نسختيْن** من أعضاء الجد ❌

مع `virtual`: الطفل يحمل **نسخة واحدة فقط** ✅

> ⚠️ فخ امتحاني: هذه المشكلة تُسمى "Diamond Problem" (مشكلة الماسة) وهي السبب الوحيد لوجود `virtual base class`

2 الحل — الكلاس الأساسي الافتراضي (Virtual Base Class)

الحل هو إضافة كلمة `virtual` عند اشتقاق `parent1` و `parent2` من `grandparent`. هكذا يضمن المُصرّف أن نسخة واحدة فقط من `grandparent` تصل لـ `child`

```
class grandparent{
    // الجد
};

class parent1 : virtual public grandparent{  // virtual هنا
    // الأب
};

class parent2 : public virtual grandparent{  // virtual هنا أيضاً
    // الأم
};

class child : public parent1, public parent2{
    // الطفل — يرث نسخة واحدة فقط من الجد
};
```

> 📌 القاعدة: الكلمتان `virtual` و `public` يمكن كتابتهما بأي ترتيب — `virtual public` أو `public virtual` — كلتاهما صحيحتان

3 مثال عملي — نظام نتائج الطلاب

هذا المثال الكلاسيكي يظهر في كل امتحان تقريباً

```text
        student
       /       \
    test      sports
       \       /
        result
```

المشكلة بدون `virtual:` `result` سيرث `r_no` مرتَيْن (من `test` ومن `sports`)

الحل:

```cpp
class student                                   // الجد — الكلاس المشترك
{
protected:
    int r_no;
public:
    void get_n(int a) { r_no = a; }
    void put_n()      { cout << "Roll No.: " << r_no << endl; }
};

class test : virtual public student             // ضروري هنا virtual
{
protected:
    int part1, part2;
public:
    void get_m(int x, int y) { part1=x; part2=y; }
    void put_m()
    {
        cout << "Part1 = " << part1 << endl;
        cout << "Part2 = " << part2 << endl;
    }
};
```

```cpp
class sports : public virtual student      // virtual ضروري هنا أيضاً
{
protected:
    int score;
public:
    void get_s(int a) { score = a; }
    void put_s() { cout << "Sports wt.: " << score << endl; }
};

class result : public test, public sports  // يرث من الاثنين
{
    int total;
public:
    void show()
    {
        total = part1 + part2 + score;     // وصول مباشر بدون تكرار
        put_n();   // نسخة واحدة من put_n
        put_m();
        put_s();
        cout << "Total = " << total << endl;
    }
};

int main()
{
    result S1;
    S1.get_n(345);
    S1.get_m(30, 35);
    S1.get_s(7);
    S1.show();
}
```

المخرج:

```text
Roll No.: 345
Part1 = 30
Part2 = 35
Sports wt.: 7
Total = 72
```

5 جدول المقارنة — مع virtual ضد بدونه

الحالة	بدون `virtual`	مع `virtual`
عدد نسخ الجد	❌ نسختان مكررتان	✅ نسخة واحدة فقط
الوصول لأعضاء الجد	❌ Ambiguity Error	✅ مباشر وواضح
أين تُكتب `virtual`	—	عند اشتقاق الآباء من الجد
ترتيب الكلمة	—	`virtual public` أو `public virtual` كلاهما صحيح
نوع المشكلة التي تحلها	—	Diamond Problem (التكرار)

6 تمرين Tracing — ما المخرج؟

```cpp
class A { protected: int x; public: void set(int a){x=a;} };

class B : virtual public A { protected: int y; public: void set(int b){y=b;} };

class C : virtual public A { protected: int z; public: void set(int c){z=c;} };

class D : public B, public C
{
public:
    void show()
    {
        cout << x + y + z << endl;
    }
};

int main()
{
    D obj;
    obj.A::set(10);  // x = 10
    obj.B::set(20);  // y = 20
    obj.C::set(30);  // z = 30
    obj.show();
}
```

المتغير	القيمة	من أين؟
`x`	10	`A::set(10)`
`y`	20	`B::set(20)`
`z`	30	`C::set(30)`
المخرج	60	`30 + 20 + 10`

🎯 أسئلة MCQ الأكثر توقعاً

سؤال (1):

> ما الهدف الرئيسي من الكلاس الأساسي الافتراضي (Virtual Base Class)؟
>
> - أ) تسريع تنفيذ البرنامج
> - ب) منع اشتقاق كلاسات جديدة
> - ج) ✅ منع تكرار (Duplication) أعضاء الكلاس الجد عند الوراثة المتعددة
> - د) السماح للكلاس المشتق بتعديل أعضاء الكلاس الأساسي

سؤال (2):

> أيٌّ من التعريفات التالية صحيح للكلاس الأساسي الافتراضي؟
>
> - أ) `class B : public virtual A` فقط
> - ب) `class B : virtual A` فقط
> - ج) ✅ `class B : virtual public A` أو `class B : public virtual A` كلاهما صحيح
> - د) `class B : virtual private A` فقط

سؤال (3):

> في مثال `result` الطالب، لو حذفنا كلمة `virtual` من `test` و `sports`، ماذا سيحدث؟
>
> - أ) البرنامج يعمل بشكل طبيعي
> - ب) Runtime Error عند التشغيل
> - ج) ✅ `r_no` — Ambiguity Error موجود مرتّيْن في `result`
> - د) يعمل لكن يطبع نتائج خاطئة

7 ترتيب البنّاءات في الوراثة — Constructors Order

هذا **المنجم الحقيقي** لأسئلة الـ Tracing. كل سؤال "ما المخرج؟" يختبر هذه القاعدة.

القاعدة الذهبية — ترتيب التنفيذ:

عند **إنشاء** الكائن ← يبدأ البنّاء من **الأب أولاً ثم الابن**

عند **تدمير** الكائن ← يبدأ المدمِّر من **الابن أولاً ثم الأب** (عكس البناء تماماً)

```text
البناء:   أب  ──►  ابن        (من فوق لتحت)
الهدم:    ابن  ──►  أب        (من تحت لفوق)
```

تمرين (1) Tracing — أبسط مثال:

```cpp
class A
{
public:
    A()  { cout << "1"; }
    ~A() { cout << "2"; }
};

class B : public A
{
public:
    B()  { cout << "3"; }
    ~B() { cout << "4"; }
};

int main()
{
    B obj;
    return 0;
}
```

الخطوة	ما يحدث	المخرج
إنشاء `obj`	بنّاء A أولاً	1
إكمال إنشاء `obj`	بنّاء B ثانياً	3
نهاية `main`	مدمِّر B أولاً	4
خروج نهائي	مدمِّر A أخيراً	2

تمرير الوسائط للأب — Parameterized Constructor:

إذا كان بنّاء الأب يأخذ قيماً، يجب على الابن تمريرها **صراحةً** عبر **قائمة التهيئة (Initialization List)**

```cpp
class Base
{
public:
    Base(int x) { cout << "Base: " << x << endl; }
};

class Derived : public Base
{
public:
    Derived(int x, int y) : Base(x)     // تمرير x صراحةً للأب
    {
        cout << "Derived: " << y << endl;
    }
};

int main()
{
    Derived obj(10, 20);
    return 0;
}
```

المخرج:

```text
Base: 10
Derived: 20
```

> ⚠️ فخ امتحاني: الصيغة `Base(x) :` تُكتب بعد رأس الدالة وقبل قوس الجسم `{`. لو نسيتها مع وجود بنّاء مُعرَّف بوسائط في الأب → Compile Error

تمرين (2) Tracing — ثلاثة مستويات:

```cpp
class A
{
public:
    A()  { cout << "A built\n"; }
    ~A() { cout << "A gone\n"; }
};

class B : public A
{
public:
    B()  { cout << "B built\n"; }
    ~B() { cout << "B gone\n"; }
};

class C : public B
{
public:
    C()  { cout << "C built\n"; }
    ~C() { cout << "C gone\n"; }
};

int main()
{
    C obj;
    return 0;
}
```

الترتيب	الحدث	المخرج
1	بنّاء A	`A built`
2	بنّاء B	`B built`
3	بنّاء C	`C built`
4	مدمِّر C	`C gone`
5	مدمِّر B	`B gone`
6	مدمِّر A	`A gone`

> 📌 القاعدة: مهما كان عدد المستويات، البنّاء دائماً من الجد للحفيد، والمدمِّر من الحفيد للجد

تمرين (3) Tracing — مع وسائط:

```cpp
class Base
{
    int x;
public:
    Base(int a) : x(a) { cout << "Base=" << x << endl; }
    ~Base()            { cout << "~Base" << endl; }
};

class Derived : public Base
{
    int y;
public:
    Derived(int a, int b) : Base(a), y(b)
    { cout << "Derived=" << y << endl; }
    ~Derived() { cout << "~Derived" << endl; }
};

int main()
{
    Derived d(5, 10);
    return 0;
}
```

المخرج:

```text
Base=5
Derived=10
~Derived
~Base
```

8 الكلاسات المجردة — Abstract Classes

ما هي الدالة الافتراضية النقية؟

الدالة الافتراضية العادية (`virtual`) تملك جسماً يمكن تنفيذه. أما الدالة الافتراضية **النقية** (`Pure Virtual`) فهي إعلان فارغ تماماً — مجرد **"وعد"** بأن الكلاسات المشتقة ستُنفّذها

الصيغة: كتابة `= 0` في نهاية الإعلان:

```cpp
class Shape
{
public:
    virtual void draw() = 0;   // ✅ دالة افتراضية نقية
    virtual double area() = 0;
};
```

تعريف الكلاس المجرد:

أي كلاس يحتوي على **دالة افتراضية نقية واحدة على الأقل** يُصبح كلاساً مجرداً تلقائياً. له قيدان لا استثناء فيهما:

- ❌ **لا يمكن إنشاء كائن** منه مباشرة
- ✅ يُستخدم **فقط كأساس (Base)** للكلاسات الأخرى

```cpp
class Shape
{
public:
    virtual void draw() = 0;    // دالة نقية → الكلاس مجرد
};

Shape s;                        // ❌ Compile Error!
// error: cannot instantiate abstract class
```

كيف نستخدمه؟ — الكلاس المشتق ينفِذ الدالة:

```cpp
class Shape
{
public:
    virtual void draw() = 0;    // "الكلاس المجرد يُعرّف "القالب
};

class Circle : public Shape
{
public:
    void draw()                 // ✅ الكلاس المشتق يُنفّذ الدالة
    {
        cout << "Drawing Circle" << endl;
    }
};

class Rectangle : public Shape
{
public:
    void draw()                 // ✅ تنفيذ مختلف لنفس الدالة
    {
        cout << "Drawing Rectangle" << endl;
    }
};

int main()
{
    // Shape s;          // ❌ ممنوع
    Circle c;            // ✅ مسموح
    Rectangle r;         // ✅ مسموح
    c.draw();            // Drawing Circle
    r.draw();            // Drawing Rectangle
}
```

ماذا لو لم يُنفِّذ الكلاس المشتق الدالة النقية؟

```cpp
class Shape    { public: virtual void draw() = 0; };

class Triangle : public Shape
{
    // نسي تنفيذ draw() !
};

Triangle t;  // ❌ Compile Error!
// Triangle أيضاً أصبح مجرداً لأنه لم يُنفِّذ الدالة النقية
```

> 📌 القاعدة: الكلاس المشتق يبقى مجرداً إذا لم يُنفِّذ جميع الدوال النقية الموروثة — وبالتالي لا يمكن إنشاء كائن منه أيضاً

جدول المقارنة — Virtual Function ضد Pure Virtual Function:

العنصر	`virtual void draw()`	`virtual void draw() = 0`
الجسم (Body)	✅ موجود	❌ غير موجود
الكلاس يصبح مجرداً؟	❌ لا	✅ نعم
يمكن إنشاء كائن؟	✅ نعم	❌ لا
إلزامية التنفيذ في المشتق	اختيارية	إلزامية
الهدف	تنفيذ قابل للتجاوز	قالب إجباري للمشتقات

سؤال MCQ متوقع (1):

> عند إنشاء كائن من كلاس مشتق يحتوي على بنّاء مُعرَّف، ما الترتيب الصحيح؟
>
> - أ) بنّاء الابن ثم بنّاء الأب
> - ب) ✅ بنّاء الأب أولاً ثم بنّاء الابن
> - ج) يعتمد على ترتيب الإعلان في main
> - د) يعمل البنّاءان في نفس الوقت

سؤال MCQ متوقع (2):

> أيٌّ من العبارات التالية صحيحة عن ترتيب المدمِّرات؟
> - أ) نفس ترتيب البنّاءات
> - ب) ✅ عكس ترتيب البنّاءات تماماً — الابن أولاً ثم الأب
> - ج) عشوائي — يعتمد على المُصرِّف
> - د) المدمِّر لا يُستدعى تلقائياً

سؤال MCQ متوقع (3):

> ما الذي يميز الكلاس المجرد (Abstract Class)؟
> - أ) لا يحتوي على أي دوال
> - ب) جميع أعضائه `static`
> - ج) ✅ يحتوي على دالة افتراضية نقية ولا يمكن إنشاء كائنات منه مباشرة
> - د) لا يمكن اشتقاق كلاسات منه

سؤال MCQ متوقع (4):

> إذا اشتقّ كلاس من كلاس مجرد لكنه لم يُنفِّذ الدوال النقية، ماذا يصبح؟
> - أ) يصبح كلاساً عادياً
> - ب) ✅ يصبح هو الآخر مجرداً ولا يمكن إنشاء كائن منه
> - ج) يُعطي Runtime Error فقط
> - د) يعمل بشكل طبيعي

سؤال MCQ متوقع (5) — True or False:

> "يمكن أن يحتوي الكلاس المجرد على دوال عادية (غير نقية) إلى جانب الدوال النقية"
> - أ) ✅ صحيح — الكلاس المجرد يمكنه امتلاك دوال عادية ومتغيرات أيضاً
> - ب) خاطئ — يجب أن تكون جميع دواله نقية